Prompt Completo para Reformulação do Sistema de Geração de Arquivos

Stack: FastAPI + Playwright + Tailwind CSS + Google Gemini 2.5

Como usar este prompt

Copie o conteúdo abaixo e cole diretamente no **Google AI Studio**, **Gemini CLI (AIOS)** ou qualquer IDE com IA. Anexe também todos os arquivos do projeto junto com o prompt.

PROMPT

Você é um engenheiro de software sênior especializado em Python, FastAPI e automação de documentos. Analise, melhore e expanda o sistema de geração de arquivos que vou te fornecer.

Contexto do Projeto

Sistema de geração de arquivos de alta qualidade (PDF, DOCX, Excel, PPTX) com API FastAPI, deployado no Railway. O sistema usa:

- **FastAPI** — framework da API REST
- **Playwright + Chromium** — renderiza HTML com Tailwind CSS e exporta como PDF de alta qualidade
- **python-docx** — geração de documentos Word profissionais
- **openpyxl** — planilhas Excel com formatação avançada

- **python-pptx** — apresentações PowerPoint com design profissional
- **Jinja2** — templates HTML dinâmicos
- **Tailwind CSS (CDN)** — estilização dos PDFs
- **Google Gemini 2.5** — geração automática de conteúdo (substitui OpenAI)

O que quero que você faça

1. Substitua completamente qualquer referência ao OpenAI pelo Google Gemini:

- Use `google-generativeai` como SDK principal
- Modelo padrão: `gemini-2.5-flash` (rápido) com opção de `gemini-2.5-pro` (mais poderoso)
- Mantenha também a compatibilidade via endpoint OpenAI do Gemini como fallback
- A chave de API deve vir da variável de ambiente `GEMINI_API_KEY`

2. Melhore a qualidade visual de todos os geradores:

- **PDF:** adicione suporte a capa, sumário automático, numeração de páginas, 5 temas de cores
- **DOCX:** melhore tabelas, adicione suporte a imagens, cabeçalho com logo, estilos consistentes
- **Excel:** adicione gráficos automáticos (pizza, linha, barra), formatação condicional, múltiplas abas
- **PPTX:** adicione mais layouts (duas colunas, imagem + texto, citação, slide de números grandes)

3. Adicione novos endpoints:

- `POST /gerar/pdf-auto` — Gemini cria o conteúdo a partir de um tópico, depois gera o PDF
- `POST /gerar/pptx-auto` — Gemini cria os slides a partir de um tópico
- `POST /gerar/excel-auto` — Gemini cria os dados da planilha a partir de um tópico
- `POST /gerar/relatorio-completo` — gera PDF + DOCX + Excel de uma vez

- `GET /listar-arquivos` — lista arquivos gerados disponíveis para download
- `DELETE /limpar-outputs` — remove arquivos antigos

4. Melhore os prompts do Gemini para cada tipo de arquivo:

- Para PDF: instrua o Gemini a usar Markdown bem estruturado com tabelas, listas e subtítulos
- Para PPTX: instrua o Gemini a retornar JSON válido com a estrutura exata dos slides
- Para Excel: instrua o Gemini a retornar JSON com cabecalhos e dados tipados corretamente
- Use temperatura 0.2-0.3 para conteúdo factual e 0.5-0.7 para conteúdo criativo

5. Adicione 5 temas de cores:

- Profissional (azul escuro — padrão)
- Corporativo (azul e verde)
- Moderno (roxo e rosa)
- Minimalista (preto e branco)
- Quente (laranja e dourado)

6. Otimize o Dockerfile para Railway:

- Multi-stage build para reduzir tamanho da imagem
- Cache de dependências Python
- Playwright com Chromium pré-instalado
- Variáveis de ambiente via `.env`

7. Adicione validação e tratamento de erros:

- Validação Pydantic em todos os endpoints
- Try/except com mensagens de erro claras em português
- Log de cada operação com timestamp

8. Crie um arquivo de teste simples:

```
# tests/test_geradores.py
# Teste básico de cada gerador sem precisar de API key
```

Estrutura esperada do projeto final

```
projeto_gerador/
├── main.py
├── requirements.txt
├── Dockerfile
├── railway.toml
├── .env.example
├── tools/
│   ├── __init__.py
│   ├── gerar_pdf.py
│   ├── gerar_docx.py
│   ├── gerar_excel.py
│   ├── gerar_pptx.py
│   └── gemini_ai.py        ← integração com Gemini
├── templates/
│   ├── base.html
│   └── relatorio.html
├── tests/
│   └── test_geradores.py
└── outputs/                ← criada automaticamente
```

Exemplo de uso esperado

```python
import requests

# 1. Gerar PDF com conteúdo automático via Gemini
response = requests.post("http://localhost:8000/gerar/pdf-auto", json={
    "topico": "Tendências de IA em 2026",
    "tipo_documento": "relatório",
    "tema": "profissional"
})
with open("relatorio_ia.pdf", "wb") as f:
    f.write(response.content)

# 2. Gerar PPTX automático
response = requests.post("http://localhost:8000/gerar/pptx-auto", json={
    "topico": "Squads de Agentes de IA",
    "num_slides": 8,
    "tema": "profissional"
})
with open("apresentacao.pptx", "wb") as f:
    f.write(response.content)

# 3. Gerar Excel automático
response = requests.post("http://localhost:8000/gerar/excel-auto", json={
    "topico": "Relatório de Vendas Q1 2026"
})
with open("vendas.xlsx", "wb") as f:
    f.write(response.content)
```

Requisitos técnicos obrigatórios

- Python 3.11+
- Google Gemini via `google-generativeai` SDK
- Playwright com Chromium (PDFs)
- Tailwind CSS via CDN nos templates HTML
- Fontes Google (Inter) carregadas nos PDFs
- Todos os arquivos em `/outputs`
- API retorna FileResponse direto para download
- CORS habilitado para integração com frontend

- Compatível com Railway (sem GPU, sem dependências pesadas)
- Comentários em português no código

Variáveis de ambiente necessárias

```
GEMINI_API_KEY=sua_chave_aqui
GEMINI_MODEL=gemini-2.5-flash
PORT=8000
```

Entregue o projeto completo e funcional, arquivo por arquivo, com todos os imports corretos e sem erros de sintaxe.